UNIVERSITÀ DELLA CALABRIA

Algorithmic Game Theory — a.a. 2026/2027 — Sebastiano A. Piccolo

Appunti in italiano, parte di `main.pdf` — 5 novembre 2026

LEZIONE 2

Teoria dell'utilità

TEORIA

Nella lezione precedente le utilità erano numeri dati ($+1$, 0, -1 per il tris). Qui si fa il passo indietro: *da dove vengono* quei numeri? La risposta è che non sono un dato primitivo, ma una **rappresentazione** delle preferenze del giocatore. Il percorso della lezione è:

1. concetti di base: alternative e preferenze;
2. rappresentare preferenze ordinali (utilità **ordinale**);
3. oltre l'ordinale: lotterie e utilità **cardinale** (VNM);
4. atteggiamento verso il rischio;
5. applicazioni e introduzione alla teoria delle decisioni.

2.1 Concetti di base

2.1.1 Alternative

Un agente sceglie all'interno di un insieme X di **alternative**, che devono essere:

- **mutuamente esclusive**: se ne realizza una sola;
- **esaustive**: coprono tutti i casi possibili.

> **Esempio (scelta dei corsi).** Le opzioni sono {Deep Learning, Algorithmic Game Theory}, ma queste *non* sono le alternative: si possono seguire entrambi i corsi o nessuno. L'insieme corretto è
>
> $$X = \{\,\mathrm{DL},\ \mathrm{AGT},\ \mathrm{DLAGT},\ \mathrm{N}\,\},$$
>
> dove DLAGT = entrambi e N = nessuno dei due.

Osservazione (è un passo di modellazione, non una formalità). Costruire X *bene* è la prima cosa da fare in un modello. Con n opzioni combinabili liberamente le alternative sono 2^n: già qui si vede che la dimensione dello spazio delle alternative è un problema *computazionale*, non solo concettuale.

2.1.2 Preferenze

Definizione 2.1 (Relazione di preferenza). Le **preferenze** sono una relazione $\succsim$ su X, cioè un sottoinsieme di $X \times X$. Si legge $x \succsim y$ come "x è *almeno buono quanto* y". La relazione è:

- **completa** se $\forall x, y \in X$ vale $x \succsim y$ oppure $y \succsim x$;
- **transitiva** se $\forall x, y, z \in X$, $[x \succsim y \text{ e } y \succsim z] \Rightarrow x \succsim z$.

Una preferenza è una **relazione di preferenza** se e solo se è completa e transitiva.

Da $\succsim$ si derivano altre due relazioni:

$$\begin{aligned} \text{preferenza } \textbf{stretta}: &\qquad x \succ y \iff [\, x \succsim y \text{ e } y \not\succsim x \,]; \\ \textbf{indifferenza}: &\qquad x \sim y \iff [\, x \succsim y \text{ e } y \succsim x \,]. \end{aligned}$$

Osservazione (completezza non vuol dire "decidere sempre"). La completezza non dice che l'agente preferisce sempre qualcosa a qualcos'altro: dice che sa sempre *confrontare* due alternative. Il caso "mi sono indifferenti" è ammesso ed è proprio $x \sim y$. Quello che la completezza esclude è l'incomparabilità ("non so dire").

2.2 Rappresentare le preferenze: utilità ordinale

Definizione 2.2 (Funzione di utilità). Una relazione di preferenza $\succsim$ è **rappresentata** da una funzione di utilità $u : X \to \mathbb{R}$ nel senso seguente:

$$x \succsim y \iff u(x) \geq u(y) \qquad \forall x, y \in X.$$

Se un giocatore trova x almeno buono quanto y, allora $u(x)$ deve essere almeno alto quanto $u(y)$. In questo senso il giocatore si comporta *come se* (*as if*) stesse massimizzando $u(\cdot)$.

Osservazione (la direzione della spiegazione). L'ultima frase va presa alla lettera e sarà ripresa alla fine della lezione: l'utilità è un *modo di descrivere* le preferenze, non la loro causa. Non si sceglie x perché ha utilità più alta; ha utilità più alta perché è ciò che si sceglie.

Teorema 2.3 (Rappresentazione ordinale). Sia X *finito*. Una preferenza può essere rappresentata da una funzione di utilità se e solo se è completa e transitiva. Inoltre, se $u : X \to \mathbb{R}$ rappresenta $\succsim$ e $f : \mathbb{R} \to \mathbb{R}$ è **strettamente crescente**, allora anche $f \circ u$ rappresenta $\succsim$.

Per l'ultima affermazione queste utilità si dicono **ordinali**: conta solo l'*informazione d'ordine*.

2.2.1 Le condizioni sono necessarie

Transitività. Sia $X = \{a, b, c\}$ e si supponga il ciclo $a \succ b \succ c \succ a$. Se esistesse u si avrebbe $u(a) > u(b) > u(c) > u(a)$, cioè $u(a) > u(a)$: assurdo.

> **Approfondimento (la *money pump*).** Filosofi ed economisti direbbero che violare la transitività genera una **pompa di soldi**. Se l'agente ha il ciclo sopra e possiede c, allora preferisce strettamente b a c: gli si può vendere b in cambio di c più un piccolo importo ε. Poi preferisce a a b: altro scambio, altro ε. Poi preferisce c ad a: terzo scambio. L'agente è tornato al punto di partenza avendo pagato 3ε, e il ciclo si può ripetere all'infinito. È l'argomento "pratico" a favore della transitività: chi la viola si fa svuotare le tasche.

Completezza. Si assuma di avere una funzione di utilità $u : X \to \mathbb{R}$. Dati due numeri $u(x)$ e $u(y)$, sono ammissibili solo due possibilità: $u(x) \geq u(y)$ oppure $u(y) \geq u(x)$. Quindi $x \succsim y$ o $y \succsim x$: la completezza è una necessità matematica, ereditata dal fatto che l'ordine $\geq$ su $\mathbb{R}$ è totale.

2.2.2 Le condizioni sono sufficienti

Dimostrazione del teorema. Sia $X = \{x_1, \ldots, x_n\}$. Grazie a completezza e transitività si possono partizionare gli elementi di X in k **classi di indifferenza** $C_1, \ldots, C_k$ (ogni classe raccoglie alternative tra loro indifferenti) ordinate in modo che $C_1 \succ C_2 \succ \ldots \succ C_k$. Si definisce allora

$$u(x) = k - i + 1 \qquad \text{per ogni } x \in C_i,$$

cioè $u = k$ sulla classe migliore, $u = k - 1$ sulla successiva, fino a $u = 1$ su C_k. Per costruzione $u(x) \geq u(y) \iff x \succsim y$. □

> **Attenzione (dove serve che X sia finito).** L'ipotesi "X finito" non è decorativa: serve a poter costruire le classi di indifferenza e a numerarle. Con X infinito completezza e transitività *non* bastano. Il controesempio classico sono le preferenze **lessicografiche** su $X = \mathbb{R}^2$, dove $(a_1, a_2) \succsim (b_1, b_2)$ se $a_1 > b_1$, oppure se $a_1 = b_1$ e $a_2 \geq b_2$: la relazione è completa e transitiva ma non ammette alcuna rappresentazione con una u a valori reali.

2.2.3 Esempi: l'utilità ordinale non è interpretabile

Con $X = \{a, b, c\}$ e preferenze $a \succ b \succ c$, tutte e tre queste funzioni rappresentano le *stesse* preferenze:

	u_1	u_2	u_3
$u(a)$	-1	30	90
$u(b)$	-2	20	2
$u(c)$	-3	10	1

Il valore numerico dell'utilità ordinale **non è interpretabile**: non dice con quale *intensità* un giocatore preferisce a a b. Con u_2 le differenze sono uguali, con u_3 il salto tra a e b sembra enorme, ma le preferenze rappresentate sono identiche.

Osservazione (perché non basta). Finché si sceglie tra alternative *certe*, l'ordine basta: si prende la migliore. Ma appena entra in gioco l'**incertezza** (una scelta dà a con probabilità 0,5 e c con probabilità 0,5) serve sapere *quanto* b sta tra a e c: serve un'utilità **cardinale**. È il problema che risolve la teoria di Von Neumann–Morgenstern.

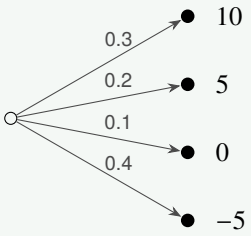
2.3 Oltre l'ordinale: le lotterie

Definizione 2.4 (Lotteria semplice). Una **lotteria semplice** è una tupla

$$L = (p_1, x_1;\ p_2, x_2;\ \ldots;\ p_n, x_n)$$

dove $x_1, \ldots, x_n \in X \subseteq \mathbb{R}$ sono **premi monetari** e $(p_1, \ldots, p_n)$ è una distribuzione di probabilità ($p_i \geq 0$, $\sum_i p_i = 1$). Si indica con $\mathcal{L}$ l'insieme delle lotterie semplici.

Esempio (una lotteria). $L = (0{,}3,\ 10;\ 0{,}2,\ 5;\ 0{,}1,\ 0;\ 0{,}4,\ -5)$.



2.3.1 Lotterie composte

Una lotteria può avere come premi altre lotterie. Siano

$$L_1 = (0{,}9:\ -1\$,\ 0{,}1:\ 5\$), \qquad L_2 = (0{,}75:\ -1\$,\ 0{,}25:\ 2\$).$$

La **lotteria composta** $L_c = (0{,}5:\ L_1,\ 0{,}5:\ L_2)$ equivale alla lotteria semplice che si ottiene moltiplicando le probabilità lungo i rami e sommando quelle che portano allo stesso premio:

$$L_e = \big(0{,}5\times0{,}9+0{,}5\times0{,}75:\ -1\$,\quad 0{,}5\times0{,}25:\ 2\$,\quad 0{,}5\times0{,}1:\ 5\$\big) = (0{,}825:\ -1\$,\ 0{,}125:\ 2\$,\ 0{,}05:\ 5\$).$$

Osservazione (ipotesi di riduzione). Questo passaggio è un'*assunzione* del modello: si suppone che al giocatore interessi solo la distribuzione finale sui premi, non il modo in cui ci si arriva (due estrazioni invece di una). Grazie a essa basta studiare $\mathcal{L}$, l'insieme delle lotterie *semplici*.

2.3.2 Valore atteso

Definizione 2.5 (Valore atteso). Per $L = (p_1, x_1; \ldots; p_n, x_n)$ il **valore atteso** è

$$\mathbb{E}(L) = \sum_{i=1}^{n} p_i\, x_i.$$

Per la lotteria dell'esempio:

$$\mathbb{E}(L) = 0{,}3 \times 10 + 0{,}2 \times 5 + 0{,}1 \times 0 + 0{,}4 \times (-5) = 2.$$

Giocando ripetutamente quella lotteria ci si aspetta di vincere in media 2.

Attenzione (il valore atteso non basta a decidere). Verrebbe da usare $\mathbb{E}$ come criterio di scelta, ma è sbagliato. Si consideri il **paradosso di San Pietroburgo**: si lancia una moneta finché non esce testa; se esce al lancio k si vincono 2^k euro. Il valore atteso è $\sum_{k\geq 1} 2^{-k} \cdot 2^k = \infty$, eppure nessuno pagherebbe più di poche decine di euro per giocare. Serve applicare l'utilità *ai premi*, non ai soldi.

2.3.3 Utilità attesa

Definizione 2.6 (Utilità attesa e funzione di Bernoulli). Data una relazione di preferenza $\succsim$ su $\mathcal{L}$, una funzione di utilità $U : \mathcal{L} \to \mathbb{R}$ è una **funzione di utilità attesa** se può essere scritta come

$$U(L) = \sum_{i=1}^{n} p_i \, u(x_i)$$

per qualche funzione $u : \mathbb{R} \to \mathbb{R}$. La funzione u si chiama **funzione di utilità di Bernoulli**.

Attenzione (due utilità diverse, due simboli diversi). u è definita sui *premi* (numeri di euro), U sulle *lotterie*. La forma di U è vincolata: deve essere **lineare nelle probabilità**. Non ogni U che rappresenta preferenze su $\mathcal{L}$ ha questa forma; il teorema VNM dice esattamente quando ce l'ha.

2.4 L'utilità di Von Neumann–Morgenstern

Von Neumann e Morgenstern (VNM) hanno fornito le condizioni sotto cui una relazione di preferenza $\succsim$ su $\mathcal{L}$ è rappresentabile da una funzione di utilità attesa.

Definizione 2.7 (Assiomi VNM).

Assioma 1 (**ordine di preferenza**) $\succsim$ è completa e transitiva.

Assioma 2 (**continuità**) se $L \succ M \succ N$, esiste $p \in [0,1]$ tale che $pL + (1-p)N \sim M$.

Assioma 3 (**indipendenza**) per ogni lotteria N e ogni $p \in [0,1]$,

$$L \succsim M \iff pL + (1-p)N \succsim pM + (1-p)N.$$

Osservazione (come leggere gli assiomi 2 e 3). La **continuità** dice che non esistono premi "infinitamente" migliori di altri: se M sta tra L e N, c'è una miscela di L e N esattamente equivalente a M. L'**indipendenza** dice che un esito *comune* alle due lotterie non deve cambiare il confronto: N compare con la stessa probabilità $1-p$ da entrambe le parti, quindi deve essere irrilevante. È l'assioma più forte dei tre e, come si vedrà con Allais, è quello che gli esseri umani violano.

Teorema 2.8 (Von Neumann–Morgenstern). Una relazione binaria $\succsim$ su $\mathcal{L}$ ammette una rappresentazione di utilità attesa se e solo se soddisfa gli assiomi 1–3. Inoltre, se U e V sono due rappresentazioni di utilità attesa di $\succsim$, allora esistono costanti $a, b \in \mathbb{R}$ con $a > 0$ tali che

$$U(\cdot) = a\,V(\cdot) + b.$$

L'ultima affermazione dice che la rappresentazione VNM è unica **a meno di trasformazioni affini**.

Osservazione (ordinale contro cardinale). È qui la differenza con il Teorema 1. Lì si poteva applicare a *u qualunque* funzione strettamente crescente; qui solo $ax + b$ con $a > 0$. Il margine di libertà è molto più stretto, e questo rende i numeri *interpretabili*: sono fissati l'origine e l'unità di misura, come per la temperatura in gradi Celsius. Ha senso dire che il salto da a a b è il doppio del salto da b a c; non ha senso dire che a vale il doppio di b. Si parla di **scala a intervalli**.

2.4.1 Necessità degli assiomi

Ordine di preferenza. Già dimostrato per il Teorema 1.

Continuità. Sia $L \succ M \succ N$; allora $u(L) > u(M) > u(N)$. Si ponga

$$p = \frac{u(M) - u(N)}{u(L) - u(N)} \in (0, 1).$$

Per la definizione di utilità attesa:

$$\begin{aligned} u\big(pL + (1-p)N\big) &= p\,u(L) + (1-p)\,u(N) \\ &= p\,u(L) + u(N) - p\,u(N) \\ &= p\big(u(L) - u(N)\big) + u(N) \\ &= \frac{u(M) - u(N)}{u(L) - u(N)}\big(u(L) - u(N)\big) + u(N) \\ &= u(M) - u(N) + u(N) = u(M). \end{aligned}$$

Quindi $pL + (1-p)N \sim M$: la continuità è necessaria.

Indipendenza. Siano $L_1 = pL + (1-p)N$ e $L_2 = pM + (1-p)N$. Per linearità dell'attesa:

$$\begin{aligned} u(L_1) &= p\,u(L) + (1-p)\,u(N), \\ u(L_2) &= p\,u(M) + (1-p)\,u(N). \end{aligned}$$

Sottraendo membro a membro, il termine comune in N si cancella:

$$u(L_1) - u(L_2) = p\big(u(L) - u(M)\big).$$

Se $L \succ M$ allora $u(L) > u(M)$, dunque $u(L_1) > u(L_2)$, cioè $L_1 \succ L_2$. L'indipendenza è necessaria.

Osservazione (da dove viene la "forza" dell'indipendenza). La cancellazione del termine $(1-p)u(N)$ funziona *perché* U è lineare nelle probabilità. Indipendenza e linearità sono in sostanza la stessa proprietà vista da due lati: uno comportamentale (l'esito comune non conta), l'altro

algebrico.

2.4.2 Sufficienza: dimostrazione del teorema VNM

La dimostrazione si divide in due parti:

- **Parte 1**: si mostra che $U(L) = \sum_i p_i\, u(x_i)$;
- **Parte 2**: si mostra che $L \succ M \iff U(L) > U(M)$ per ogni $L, M \in \mathcal{L}$.

Parte 1. Si supponga di avere n esiti $o_1, \ldots, o_n$. Per completezza e transitività si possono ordinare dal peggiore al migliore: $o_1 \precsim \ldots \precsim o_n$ (nel senso o_n migliore). Si fissano i due estremi della scala:

$$u(o_1) = 0, \qquad u(o_n) = 1.$$

Si definisce la famiglia di lotterie semplici

$$L(p) = p \cdot o_n + (1 - p) \cdot o_1,$$

cioè "il premio migliore con probabilità p, il peggiore con probabilità $1 - p$". Per l'assioma di continuità, per ogni esito o_i esiste una probabilità $q_i \in [0, 1]$ tale che $L(q_i) \sim o_i$; si pone $u(o_i) = q_i$. Ne segue che l'utilità di una lotteria $M = \sum_i p_i o_i$ è l'attesa di u:

$$u(M) = u\Big(\sum_i p_i o_i\Big) = \sum_i p_i\, u(o_i) = \sum_i p_i\, q_i.$$

Osservazione (che cosa è davvero q_i). q_i è un modo operativo di *misurare* l'utilità: è la probabilità di vincere il premio migliore che rende il giocatore indifferente a ricevere o_i con certezza. È una definizione che si può realizzare come esperimento, chiedendo alla persona di aggiustare q finché non è indifferente, ed è esattamente questo che rende la scala cardinale invece che ordinale: q_i posiziona o_i *tra* i due estremi.

Parte 2. Per l'assioma di continuità il giocatore è indifferente tra l'esito o_i e la lotteria $q_i \cdot o_n + (1 - q_i) \cdot o_1$. Insieme all'assioma di indipendenza (si sostituisce un esito alla volta dentro la lotteria) il giocatore è indifferente tra le due lotterie

$$M = \sum_i p_i o_i \qquad \text{e} \qquad M' = \sum_i p_i \big[\, q_i \cdot o_n + (1 - q_i) \cdot o_1 \,\big].$$

Ma M' coinvolge solo i due esiti estremi, quindi si può riscrivere come lotteria semplice:

$$M' = \Big(\sum_i p_i q_i\Big) \cdot o_n + \Big(\sum_i p_i (1 - q_i)\Big) \cdot o_1 = U(M) \cdot o_n + \big(1 - U(M)\big) \cdot o_1.$$

Ogni lotteria è dunque equivalente a una lotteria sui soli due estremi, la cui unica caratteristica è la probabilità $U(M)$ di ottenere il premio migliore.

Si supponga ora $L \succ M$ e si definiscano

$$L' = U(L) \cdot o_n + \big(1 - U(L)\big) \cdot o_1,$$
$$M' = U(M) \cdot o_n + \big(1 - U(M)\big) \cdot o_1.$$

Vale l'ordine $L' \sim L \succ M \sim M'$, quindi $L' \succ M'$. Ma L' e M' differiscono solo per la probabilità del premio migliore, e tra due lotterie di questa famiglia si preferisce quella con la probabilità più alta: dunque $U(L) > U(M)$. Vale anche il viceversa, quindi $L \succ M \iff U(L) > U(M)$. □

2.5 Il paradosso di Allais

Si risponda d'istinto a due domande. *Prima*: quale lotteria si preferisce?

$$L = (1\text{: } 1.000.000\$), \qquad M = (0{,}89\text{: } 1.000.000\$,\ \ 0{,}10\text{: } 5.000.000\$,\ \ 0{,}01\text{: } 0\$).$$

Seconda: e tra queste?

$$N = (0{,}89\text{: } 0\$,\ \ 0{,}11\text{: } 1.000.000\$), \qquad O = (0{,}90\text{: } 0\$,\ \ 0{,}10\text{: } 5.000.000\$).$$

La maggior parte delle persone sceglie L nella prima coppia (il milione sicuro) e O nella seconda. Questa combinazione è **incompatibile** con la teoria dell'utilità attesa.

Approfondimento (perché è una contraddizione). Si normalizzi $u(0) = 0$ e si scrivano $u_1 = u(1\text{M})$, $u_5 = u(5\text{M})$.

- $L \succ M$ significa $u_1 > 0{,}89\,u_1 + 0{,}10\,u_5$, cioè $0{,}11\,u_1 > 0{,}10\,u_5$;
- $O \succ N$ significa $0{,}10\,u_5 > 0{,}11\,u_1$.

Le due disuguaglianze si contraddicono: *nessuna* funzione di utilità u può giustificare entrambe le scelte.
Visto con l'assioma di indipendenza è ancora più netto. Si scriva ogni lotteria come miscela con $p = 0{,}11$:

	parte con probabilità 0,11	parte con probabilità 0,89
L	1M certo	1M
M	$(\frac{10}{11}\text{: } 5\text{M},\ \frac{1}{11}\text{: } 0)$	1M
N	1M certo	0
O	$(\frac{10}{11}\text{: } 5\text{M},\ \frac{1}{11}\text{: } 0)$	0

Le coppie (L, M) e (N, O) differiscono nella prima colonna esattamente allo stesso modo; la seconda colonna è un esito *comune* (dentro ogni coppia). Per l'indipendenza l'esito comune non deve contare, quindi chi preferisce L a M deve preferire N a O.

Osservazione (che cosa insegna il paradosso). Allais non dimostra che la teoria VNM è sbagliata: dimostra che è **normativa** e non **descrittiva**. Dice come si *dovrebbe* decidere per essere coerenti, non come le persone decidono davvero. L'effetto in gioco si chiama *certainty effect*: il salto da 0,99 a 1 (certezza) pesa psicologicamente molto più del salto da 0,10 a 0,11. Le teorie descrittive alternative (per esempio la *prospect theory* di Kahneman e Tversky) nascono proprio da qui.

2.6 Atteggiamento verso il rischio

Definizione 2.9 (Lotteria equa). Una lotteria $L = (p\text{: } x,\ (1-p)\text{: } y)$ è **equa** se il suo valore atteso è nullo:

$$\mathbb{E}(L) = p\,x + (1-p)\,y = 0.$$

Attenzione (notazione delle slide). Le slide scrivono "$L = p \cdot x + (1-p) \cdot y = 0$". La lotteria L è un oggetto (una distribuzione), non un numero: a essere nullo è il suo *valore atteso* $\mathbb{E}(L)$. Si tratta di un abuso di notazione, ma la sostanza è "giocare questa lotteria non costa e non rende nulla, in media".

Definizione 2.10 (Atteggiamenti verso il rischio). Un giocatore con funzione di utilità di Bernoulli u è:

- **neutrale al rischio** se u è **lineare**, $u(x) = ax + b$ (con $a > 0$): è indifferente di fronte alle lotterie eque;
- **avverso al rischio** se u è **concava**: se strettamente concava, non gioca *nessuna* lotteria equa;
- **amante del rischio** (*risk seeking*) se u è **convessa**: se strettamente convessa, gioca *tutte* le lotterie eque.

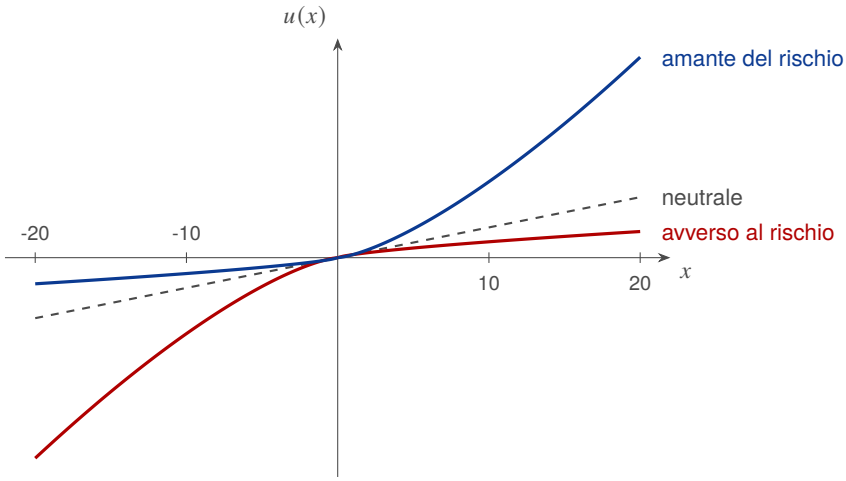


Figura 2.1: Le tre forme della funzione di utilità sui guadagni. L'avverso al rischio soffre una perdita più di quanto goda un guadagno della stessa entità; per l'amante del rischio vale il contrario.

Approfondimento (perché concavità significa avversione al rischio). È la **disuguaglianza di Jensen**. Se u è concava, per ogni lotteria L

$$\mathbb{E}\big[u(L)\big] \;\le\; u\big(\mathbb{E}[L]\big),$$

cioè l'utilità del gioco è al più l'utilità di incassare con certezza il valore atteso. Chi è avverso al rischio preferisce quindi sempre il certo all'aleatorio di pari media; per una lotteria equa aggiunta alla propria ricchezza, non giocare è almeno altrettanto buono che giocare.
Si definisce **equivalente certo** (*certainty equivalent*) $CE(L)$ il premio certo che dà la stessa utilità della lotteria, cioè $u(CE(L)) = \mathbb{E}[u(L)]$. La differenza

$$\mathbb{E}[L] - CE(L)$$

è il **premio per il rischio**: quanto il giocatore è disposto a rinunciare per eliminare l'incertezza. Per un avverso al rischio è positivo, ed è esattamente la quantità su cui si regge il mercato assicurativo.

2.7 Applicazioni

In entrambe le applicazioni la funzione di Bernoulli è $u(x) = \sqrt{x}$, che è strettamente concava: il giocatore è avverso al rischio.

2.7.1 Condivisione del rischio (*risk sharing*)

Due giocatori avversi al rischio, con $u(x) = \sqrt{x}$, possiedono due attività rischiose *indipendenti*

$$A_1 = A_2 = (0{,}5,\ 100;\ 0{,}5,\ 0).$$

L'utilità di ciascuna attività, presa da sola, è

$$u(A_1) = u(A_2) = 0{,}5 \times \sqrt{100} + 0{,}5 \times \sqrt{0} = 5.$$

Se i due giocatori formano un **fondo comune** in cui ciascuno possiede metà del totale, ogni giocatore si ritrova con l'attività

$$A_m = (0{,}25,\ 100;\ 0{,}5,\ 50;\ 0{,}25,\ 0),$$

la cui utilità è

$$u(A_m) = 0{,}25 \times \sqrt{100} + 0{,}5 \times \sqrt{50} + 0{,}25 \times \sqrt{0} \approx 2{,}5 + 3{,}54 = 6{,}04 > 5.$$

Entrambi i giocatori stanno **meglio** pur non avendo cambiato il valore atteso.

> **Approfondimento (da dove viene A_m e perché funziona).** Le due attività sono indipendenti, quindi i quattro casi (vince A_1 sì/no, A_2 sì/no) hanno probabilità 0,25 ciascuno. Il montante totale è 200, 100, 100, 0; metà del fondo vale quindi 100, 50, 50, 0, da cui $A_m = (0{,}25, 100;\ 0{,}5, 50;\ 0{,}25, 0)$. Il valore atteso è rimasto 50 in entrambi i casi: quello che è cambiato è la *varianza*, scesa da 2500 a 1250. Mettere insieme rischi indipendenti li **diversifica**, e per un giocatore con u concava meno varianza a parità di media significa più utilità. È il principio su cui si reggono i fondi comuni di investimento e le mutue assicurative.

2.7.2 Assicurazione

Un giocatore avverso al rischio con $u(x) = \sqrt{x}$ possiede l'attività $A = (0{,}5,\ 100;\ 0{,}5,\ 0)$. Esiste una compagnia di assicurazione **neutrale al rischio** con molto capitale.

Quanto è disposto a pagare il giocatore? Il giocatore paga un premio P e in cambio riceve con certezza 100; il suo patrimonio certo è quindi $100 - P$. Accetta se questo gli dà almeno l'utilità della lotteria:

$$u(100 - P) \ \geq\ 0{,}5 \times u(100) + 0{,}5 \times u(0) \implies \sqrt{100 - P} \geq 5 \implies 100 - P \geq 25 \implies P \leq 75.$$

Quanto vuole la compagnia? Essendo neutrale al rischio, le basta che il premio copra il valore atteso del risarcimento:

$$P \ \geq\ 0{,}5 \times 100 + 0{,}5 \times 0 \implies P \geq 50.$$

Conclusione. Entrambe le parti guadagnano se il contratto si stipula a un premio

$$P \in [50,\ 75].$$

Osservazione (lettura con l'equivalente certo). $\sqrt{100 - P} \geq 5$ significa $100 - P \geq 25$: l'equivalente certo della lotteria è $CE(A) = 25$, mentre il suo valore atteso è $\mathbb{E}(A) = 50$. Il premio per il rischio è $50 - 25 = 25$, ed è esattamente l'ampiezza dell'intervallo $[50, 75]$: il **surplus** che le due parti si dividono. L'assicurazione crea valore perché sposta il rischio da chi lo odia a chi gli è indifferente.

Attenzione (il patrimonio iniziale conta). Qui si è assunto implicitamente che il giocatore non abbia altra ricchezza: l'utilità si calcola su $100 - P$ e su 100 o 0. Se avesse un patrimonio iniziale w, le utilità sarebbero $u(w + 100 - P)$ e $u(w + 100)$, $u(w)$, e il premio massimo accettabile sarebbe diverso (più basso in termini relativi: più si è ricchi, meno si è avversi al rischio con $u(x) = \sqrt{x}$). La funzione di Bernoulli si applica alla *ricchezza finale*, non al guadagno della singola scommessa.

2.8 Riepilogo sulla teoria VNM

- La teoria VNM fornisce un modo per costruire funzioni di utilità **cardinali**: i valori numerici sono interpretabili su una **scala a intervalli** e, in certe situazioni, su una scala di rapporti.
- La teoria VNM fornisce una teoria **normativa** della decisione: massimizzare l'utilità attesa. È molto usata in applicazioni finanziarie ed economiche.
- La teoria VNM **non descrive** il comportamento umano. Un'alternativa non è preferita perché ha un'utilità più alta; piuttosto, le si associa un'utilità più alta perché è l'alternativa preferita.

2.9 Laboratorio: Battaglia navale e ricerca bayesiana

LAB

[Battleship_Blank.ipynb, Lab_2_Battleship.pdf]

Obiettivo: costruire un'IA per la **battaglia navale**, partendo da strategie euristiche semplici e arrivando a una strategia **adattiva** che a ogni turno spara dove l'*utilità attesa* è massima. Quest'ultima tecnica si chiama **ricerca bayesiana** (*Bayesian search*) ed è uno degli algoritmi più efficaci per i problemi di ricerca e di "caccia al tesoro".

Il collegamento con la teoria della lezione è diretto: non c'è un avversario strategico, ma c'è **incertezza**. Si deve scegliere l'azione (la cella su cui sparare) che massimizza l'utilità attesa, dove la probabilità viene aggiornata a ogni colpo con le informazioni raccolte.

2.9.1 Regole e impostazione

Il campo è una griglia 10×10 su cui sono disposte cinque navi, in orizzontale o in verticale, senza sovrapposizioni e senza uscire dal bordo:

Nave	Celle
Portaerei (*Carrier*)	5
Corazzata (*Battleship*)	4
Incrociatore (*Cruiser*)	3
Sommergibile (*Submarine*)	3
Cacciatorpediniere (*Destroyer*)	2
Totale	**17**

Non essendoci un avversario vero, le strategie non si valutano in partite vinte ma in **numero di turni necessari ad affondare tutte le navi**: 17 è il minimo teorico (nessun colpo a vuoto), 100 il massimo.

Lo stato della conoscenza si tiene in una matrice `shot_grid` con `0` = cella inesplorata, `1` = colpo a vuoto, `2` = colpo a segno.

> **Attenzione (un bug nel generatore fornito).** La funzione `create_random_disposition` del notebook estrae le coordinate così:
>
> ```
> if horizontal:
>     x = np.random.randint(0, grid_size - 1)        # riga
>     y = np.random.randint(0, grid_size - length)   # colonna di partenza
> else:
>     x = np.random.randint(0, grid_size - length)
>     y = np.random.randint(0, grid_size - 1)
> ```
>
> `np.random.randint(0, n)` esclude `n`. Quindi per una nave orizzontale la riga sta in $[0, 8]$ (la riga 9 non esce mai) e la colonna di partenza in $[0,\ 9 - \texttt{length}]$, cioè l'ultima cella occupata arriva al più alla colonna 8. Simmetricamente per le navi verticali. Il risultato è che **l'ultima riga e l'ultima colonna non sono mai occupate**: su 500 partite generate, le celle della riga 9 e della colonna 9 non contengono mai una nave, mentre con il generatore corretto sono occupate rispettivamente 642 e

660 volte. Gli estremi corretti sarebbero `randint(0, grid_size)` e `randint(0, grid_size - length + 1)`.
Conseguenza pratica: con il generatore del notebook il 19% del campo è di fatto acqua garantita e le cifre misurate sono un po' ottimistiche. Più sotto i risultati sono riportati con entrambi i generatori: la differenza arriva a circa 3 turni.

2.9.2 Strategia 1: tiro casuale

La strategia più semplice: si spara a caso, ignorando del tutto gli esiti. Non è una vera strategia, ma serve da **riferimento** (*baseline*): battere il caso è il minimo sindacale.

```python
def random_attack_strategy(grid):
    shot_grid = np.zeros((10, 10), dtype=int)
    cells = [(i, j) for i in range(10) for j in range(10)]
    np.random.shuffle(cells)
    hits = turns = 0
    for (i, j) in cells:
        shot_grid[i, j] = 2 if grid[i, j] else 1
        turns += 1
        if grid[i, j]:
            hits += 1
            if hits == 17:              # tutte le navi affondate
                break
    return shot_grid, turns
```

Osservazione (mescolare invece di estrarre). Sparare "a caso" si può implementare estraendo una cella e riprovando se è già stata colpita, ma verso la fine della partita quasi tutte le celle sono esplorate e il ciclo di riprova diventa lento. Mescolare una volta la lista delle 100 celle e scorrerla dà la stessa distribuzione senza riprove.

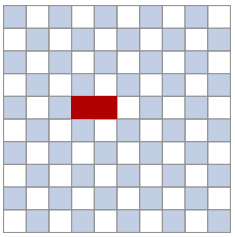
2.9.3 Strategia 2: *hunt and target*

La strategia casuale va male perché tratta tutte le celle come equiprobabili e *butta via* l'informazione dei colpi a segno. L'idea di **hunt and target** è alternare due modalità:

- **hunt** (caccia): si cerca a caso finché non si colpisce qualcosa;
- **target** (bersaglio): appena si colpisce, si spara nelle celle adiacenti (sopra, sotto, destra, sinistra) non ancora esplorate, finché la nave è affondata.

È il classico compromesso tra **esplorazione** e **sfruttamento**.

In più, in modalità hunt non serve sparare ovunque: la nave più piccola occupa 2 celle, quindi una nave non può nascondersi dentro una sola "casella nera" di una **scacchiera**. Basta sparare sulle celle con $(i + j)$ pari: 50 celle invece di 100, con la garanzia di intercettare comunque ogni nave.



Le celle della scacchiera (in blu) bastano: qualunque nave da 2 celle (in rosso) ne tocca almeno una.

```python
def hunt_and_target_strategy(grid):
    shot_grid = np.zeros((10, 10), dtype=int)
    hunt = [(i, j) for i in range(10) for j in range(10) if (i + j) % 2 == 0]
    np.random.shuffle(hunt)
    targets = []                        # pila delle celle da verificare
    hits = turns = 0
    while hits < 17:
        if targets:
            i, j = targets.pop()
        else:
            i, j = hunt.pop()
        if shot_grid[i, j]:             # gia' esplorata: salta il turno
            continue
        shot_grid[i, j] = 2 if grid[i, j] else 1
        turns += 1
        if grid[i, j]:
            hits += 1
            for di, dj in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                a, b = i + di, j + dj
                if 0 <= a < 10 and 0 <= b < 10 and shot_grid[a, b] == 0:
                    targets.append((a, b))
    return shot_grid, turns
```

Attenzione (la pila può contenere celle già colpite). Una cella può finire nella pila `targets` due volte (da due colpi a segno diversi), oppure essere colpita nel frattempo. Il controllo `if shot_grid[i, j]: continue` prima di sparare è indispensabile: senza, si conterebbero turni per colpi mai sparati e il risultato sarebbe gonfiato.

Approfondimento (si può fare meglio in modalità target). L'implementazione sopra è "la più semplice": spara alle quattro celle adiacenti senza ragionare. Un miglioramento classico è sfruttare l'**orientamento**: dopo due colpi a segno allineati si sa che la nave è orizzontale o verticale, quindi si possono scartare le celle adiacenti nell'altra direzione. La ricerca bayesiana della prossima sezione ottiene questo effetto *automaticamente*, senza regole scritte a mano.

2.9.4 La teoria della ricerca bayesiana

[Lab_2_Battleship.pdf]

L'idea della strategia 3 viene dalla **teoria della ricerca bayesiana** (*Bayesian search theory*), nata negli anni '40 per ritrovare relitti di navi, aerei e sottomarini. Si sovrappone una griglia all'area in cui si trova

l'oggetto e si guida la ricerca calcolando e ricalcolando la posizione *più probabile*: è un algoritmo iterativo e adattivo basato sul **massimo a posteriori**.

Definizione 2.11 (Ricerca bayesiana, schema generale).

1. **Inizializzazione**: si calcola la distribuzione a priori con l'informazione disponibile.
2. **A ogni iterazione**:
   - *scelta dell'azione*: si sceglie la cella di massima utilità;
   - *osservazione*: si esegue l'azione e se ne osserva l'esito;
   - *aggiornamento*: si aggiorna la distribuzione a posteriori in base all'esito.

Il caso base: un oggetto in n scatole

Ci sono n scatole $S = \{1, \dots, n\}$ e un solo oggetto. Il vettore a priori è $\mathbf{p}^{(0)} = (p_1^{(0)}, \dots, p_n^{(0)})$, dove $p_i^{(0)}$ è la probabilità che l'oggetto sia nella scatola i. Ispezionare la scatola i costa c_i (tempo o denaro) e la ricerca è **imperfetta**: se l'oggetto è in i, ispezionando i lo si trova con probabilità $q_i \in (0, 1]$ (**probabilità di rilevazione**).

Si supponga di ispezionare al passo t la scatola k senza trovare nulla (evento F_k). La probabilità di questo evento è $\Pr(F_k) = 1 - p_k^{(t)} q_k$, e con il teorema di Bayes:

$$\text{scatola ispezionata } k: \quad p_k^{(t+1)} = \Pr(\text{oggetto in } k \mid F_k) = \frac{\Pr(F_k \mid \text{oggetto in } k)\, p_k^{(t)}}{\Pr(F_k)} = \frac{(1 - q_k)\, p_k^{(t)}}{1 - p_k^{(t)} q_k},$$

$$\text{altre scatole } j \neq k: \quad p_j^{(t+1)} = \Pr(\text{oggetto in } j \mid F_k) = \frac{p_j^{(t)}}{1 - p_k^{(t)} q_k}.$$

Osservazione (come leggere l'aggiornamento). Un'ispezione fallita *abbassa* la probabilità della scatola ispezionata (non la azzera, se $q_k < 1$: l'oggetto potrebbe esserci e non essere stato visto) e *alza* in proporzione quella di tutte le altre. Il denominatore comune serve solo a rinormalizzare: le probabilità tornano a sommare 1.

Definizione 2.12 (Indice di ricerca). L'utilità di ispezionare la scatola i al tempo t è la probabilità di trovare l'oggetto *per unità di costo*:

$$I_i^{(t)} = \frac{p_i^{(t)} q_i}{c_i}.$$

Teorema 2.13 (ottimalità della ricerca golosa). Quando si cerca *un solo* oggetto in n scatole *indipendenti*, la strategia **golosa** (*greedy*) che a ogni passo ispeziona la scatola con l'indice $I_i^{(t)}$ più alto è ottimale.

Esempio (il volo Air France 447). Il 1° giugno 2009 il volo AF 447 (Rio de Janeiro–Parigi) scomparve sull'Atlantico. Le slide mostrano l'area di ricerca: l'ultima posizione nota, la rotta prevista e un cerchio di 40 miglia nautiche attorno, con il prior e un posteriore disegnati sulla griglia. Secondo il paper di Stone et al. incluso nel materiale, il prior era una *miscela* di tre distribuzioni: uni-

> forme nel cerchio, basata sui dati di incidenti precedenti, e ottenuta ripercorrendo all'indietro la deriva dei rottami recuperati. Le campagne di ricerca degli anni precedenti, fallite, sono state usate come ispezioni senza successo per aggiornarlo, tenendo conto anche della possibilità che i segnalatori acustici delle scatole nere non avessero funzionato. Nel 2011 la ricerca cominciò dal centro del posteriore e trovò il relitto in una settimana. [Search_for_the_Wreckage_of_Air_France_Flight_AF_447.pdf]

2.9.5 Strategia 3: ricerca bayesiana per la battaglia navale

Nella battaglia navale i costi sono uniformi ($c_i = 1$) e la rilevazione è perfetta ($q_i = 1$: se si spara su una nave la si colpisce). Ma ci sono **5 bersagli** e le caselle **non sono indipendenti** (una nave occupa celle contigue): l'ottimalità della strategia golosa non vale più. La ricerca bayesiana resta però un'ottima euristica:

1. si calcola una mappa di probabilità sulla griglia per *ciascuna* nave (5 mappe di densità);
2. le si somma in un'unica mappa (il *prior*);
3. a ogni colpo si sa se è a segno o a vuoto e si aggiorna la mappa (il *posteriore*).

Come si costruisce la mappa. Per ogni cella e per ogni nave **non ancora affondata** si contano le **configurazioni valide**, cioè le pose (orizzontali o verticali) della nave che coprono quella cella senza uscire dal bordo e senza toccare celle incompatibili. Su un campo vuoto il conteggio favorisce il centro: una cella d'angolo è coperta da poche pose, una centrale da molte. Sommando le cinque navi si ottiene il prior:

10	15	19	21	22	22	21	19	15	10
15	20	24	26	27	27	26	24	20	15
19	24	28	30	31	31	30	28	24	19
21	26	30	32	33	33	32	30	26	21
22	27	31	33	34	34	33	31	27	22
22	27	31	33	34	34	33	31	27	22
21	26	30	32	33	33	32	30	26	21
19	24	28	30	31	31	30	28	24	19
15	20	24	26	27	27	26	24	20	15
10	15	19	21	22	22	21	19	15	10

Prior: numero di pose valide che coprono ogni cella, sommato sulle cinque navi (calcolato col codice sotto).

> **Osservazione (da dove vengono 10 e 34).** Nell'angolo $(0, 0)$ ogni nave ha esattamente 2 pose che lo coprono (una orizzontale, una verticale): $5 \times 2 = 10$. Al centro una nave da ℓ celle ha ℓ pose orizzontali e ℓ verticali: $2(5 + 4 + 3 + 3 + 2) = 34$. Il rapporto 3,4 tra centro e angolo è il motivo per cui conviene cominciare dal centro.

Aggiornamento dopo un colpo a vuoto. Una nave non può stare su una cella risultata vuota: si scartano tutte le pose che la coprono e si ricontano.

Aggiornamento dopo un colpo a segno. Si entra in **modalità target** per affondare la nave colpita: si contano *solo* le configurazioni valide che **passano per il colpo a segno**. Si resta in modalità target finché c'è almeno una nave colpita ma non ancora affondata.

Affondamento. Quando una nave affonda, l'avversario dichiara quale: la si toglie dall'elenco di quelle da cercare e le sue celle smettono di contare come colpi a segno "aperti".

```python
SHIPS = {1: 5, 2: 4, 3: 3, 4: 3, 5: 2}       # id nave -> lunghezza
UNEXPLORED, MISS, HIT, SUNK = 0, 1, 2, 3

def placements(length):
    out = []
    for horizontal in (True, False):
        for i in range(10 if horizontal else 10 - length + 1):
            for j in range(10 - length + 1 if horizontal else 10):
                out.append([(i, j + k) if horizontal else (i + k, j)
                            for k in range(length)])
    return out

PLACEMENTS = {L: placements(L) for L in set(SHIPS.values())}

def likelihood_map(shot, remaining):
    post = np.zeros((10, 10))
    targeting = (shot == HIT).any()          # c'e' una nave colpita e non affondata
    for length in remaining:                 # solo le navi ancora a galla
        for cells in PLACEMENTS[length]:
            if any(shot[a, b] in (MISS, SUNK) for a, b in cells):
                continue                     # posa incompatibile
            n_hit = sum(shot[a, b] == HIT for a, b in cells)
            if targeting and n_hit == 0:
                continue                     # in target: solo pose per i colpi
            for a, b in cells:
                if shot[a, b] == UNEXPLORED:
                    post[a, b] += n_hit if targeting else 1
    return post

def bayesian_search_strategy(grid):
    shot = np.zeros((10, 10), dtype=int)
    remaining = list(SHIPS.values())
    hits_on = {sid: 0 for sid in SHIPS}
    turns = 0
    while remaining:
        post = likelihood_map(shot, remaining)
        i, j = np.unravel_index(np.argmax(post), post.shape)
        turns += 1
        sid = grid[i, j]
        if sid == 0:
            shot[i, j] = MISS
            continue
        shot[i, j] = HIT
        hits_on[sid] += 1
        if hits_on[sid] == SHIPS[sid]:       # affondata: l'avversario lo dichiara
            shot[grid == sid] = SUNK
            remaining.remove(SHIPS[sid])
    return shot, turns
```

Osservazione (il peso in modalità target). Ogni posa che passa per h colpi a segno viene contata h volte, una per ciascun colpo attraverso cui "passa". È esattamente quello che si ottiene seguendo alla lettera le slide ("per ogni colpo a segno, conta le configurazioni che ci passano") e ha l'effetto desiderato: dopo due colpi allineati, le pose lungo quella linea pesano doppio e la strategia prosegue nella direzione giusta senza regole scritte a mano.

Osservazione (dov'è il teorema di Bayes). Il nome "bayesiana" è giustificato anche se nel codice non compare la formula. Il prior è uniforme su tutte le pose di ogni nave; una cella risultata a vuoto ha verosimiglianza 0 per ogni posa che la occupa, quindi tutte quelle pose vengono eliminate dal posteriore. Contare le pose sopravvissute è calcolare la probabilità a posteriori marginale di ogni cella, a meno della costante di normalizzazione, che non serve perché si cerca solo l'*argmax*. È il caso $q_i = 1$ dell'aggiornamento visto sopra: una ricerca fallita azzera la cella.

Attenzione (le navi si contano una alla volta, non insieme). Il conteggio tratta ogni nave come se fosse sola sul campo: due navi possono "sovrapporsi" nel conteggio anche se nella realtà non possono. Il calcolo esatto richiederebbe di enumerare le disposizioni *congiunte* delle cinque navi compatibili con tutti i dati, che sono troppe per essere contate a ogni turno (vedi l'analisi informativa più sotto). Sommare le mappe delle singole navi è un'approssimazione: è il motivo per cui le slide la chiamano *euristica*.

Attenzione (senza le navi affondate il programma si blocca). Una prima versione "ingenua" di questa strategia, che *non* tiene traccia delle navi affondate e cerca sempre tutte e cinque, entra in **ciclo infinito**. Succede così: una nave da 2 celle viene colpita tutta e si trova circondata da celle già esplorate. I suoi colpi a segno restano "aperti", quindi si è in modalità target; ma l'unica posa che li spiega non tocca alcuna cella inesplorata e quindi non assegna voti a nessuno. Se tutti i colpi aperti sono in questa condizione il posteriore è *tutto zero*, `argmax` restituisce sempre $(0, 0)$ e si spara all'infinito sulla stessa cella.
Con la traccia delle navi affondate il problema scompare da solo: un colpo a segno aperto appartiene sempre a una nave ancora a galla, e la posa *vera* di quella nave è valida e tocca almeno una cella inesplorata, quindi il posteriore non può essere nullo. Nella simulazione lo si è controllato con un `assert post[i, j] > 0` a ogni turno, su tutte le partite.

Approfondimento (una semplificazione della simulazione). Quando una nave affonda, il codice marca come `SUNK` esattamente le sue celle leggendole dalla griglia vera (`shot[grid == sid] = SUNK`). Un giocatore reale sa *quale* nave è affondata ma non sempre *quali* dei suoi colpi a segno le appartenessero (due navi possono toccarsi). Di solito lo si deduce, ma non sempre: la simulazione è quindi leggermente ottimistica rispetto al gioco vero.

2.9.6 Risultati misurati

Le quattro strategie (le tre del notebook più la versione bayesiana senza navi affondate, per confronto) sono state eseguite sulle *stesse* 500 partite, generate sia con il generatore del notebook sia con quello corretto.

	Casuale	Hunt and target	Bayes senza affondate	Bayes con affondate
Generatore del notebook (riga 9 e colonna 9 sempre vuote)				
Turni medi	95,2	60,9	54,6	**41,3**
Deviazione standard	5,1	9,1	9,4	9,0
Mediana	97	62	54	41
Minimo – massimo	68–100	34–79	27–77	23–67
Partite entro 50 turni	0%	16,0%	33,8%	84,4%
Generatore corretto				
Turni medi	95,6	61,3	56,1	**44,3**
Deviazione standard	4,4	8,9	8,6	8,4
Mediana	97	62	56	43
Minimo – massimo	76–100	31–80	31–78	27–68
Partite entro 50 turni	0%	12,0%	25,0%	76,8%

- Il primo salto (dal caso a *hunt and target*) fa risparmiare circa 35 turni.
- Il secondo è la ricerca bayesiana *fatta bene*: altri 20 turni in meno, con oltre tre partite su quattro chiuse entro 50 colpi. I 41–44 turni misurati sono in linea con i 40–44 dichiarati nelle slide del docente.
- Senza la traccia delle navi affondate la ricerca bayesiana guadagna solo 6 turni su hunt and target: **l'informazione dell'affondamento vale da sola circa 13 turni**.
- Il bug del generatore abbassa di circa 3 turni la media della ricerca bayesiana (41,3 contro 44,3) e di meno di un turno quella di hunt and target. Con le navi confinate in una griglia 9×9 le configurazioni possibili sono meno, quindi c'è meno informazione da scoprire: ne approfitta di più la strategia che sfrutta meglio l'informazione.

2.9.7 Analisi in teoria dell'informazione

Quanta informazione c'è in una partita. Su una griglia 10×10 una nave da 2 celle si può posare in $2 \times 10 \times 9 = 180$ modi, una da 3 in 160, una da 4 in 140, una da 5 in 120. Le configurazioni possibili delle cinque navi sono quindi

$$\Omega = 180 \times 160 \times 160 \times 140 \times 120 = 7{,}74 \cdot 10^{10} \approx 2^{36{,}17}.$$

Circa metà sono inammissibili perché le navi si sovrappongono: le configurazioni legali sono circa 2^{35}, quindi una disposizione contiene circa **35 bit** di informazione.

Quanta ne porta un colpo. L'informazione media di un colpo è la sua **entropia di Shannon**, $H = -\sum_i p_i \log_2 p_i$. Se si colpisce una nave con probabilità $17/100$ e si manca con probabilità $83/100$:

$$H(\text{colpo}) = -\left(\tfrac{17}{100}\log_2\tfrac{17}{100} + \tfrac{83}{100}\log_2\tfrac{83}{100}\right) \approx 0{,}66 \text{ bit}, \qquad \mathbb{E}(\#\text{colpi}) \approx \frac{35 \text{ bit}}{0{,}66 \text{ bit/colpo}} \approx 53.$$

La ricerca bayesiana ne impiega 40–44, il 17% in meno: estrae circa $35/42 \approx 0{,}8$ bit per colpo, perché sceglie le celle in modo da *livellare* il posteriore e rendere ogni tiro il più informativo possibile.

Attenzione (53 non è un limite inferiore). Le slide dicono che la ricerca bayesiana "migliora il limite del 17%". Un limite che si può battere non è un limite: 53 è il numero di colpi che servirebbero se *ogni* colpo avesse probabilità di successo $17/100$, cioè se fosse informativo quanto un tiro a caso. Un buon giocatore sceglie celle con probabilità di successo più vicina a $1/2$, e un colpo con due esiti porta al più 1 bit quando i due esiti sono equiprobabili. Il vero limite informativo è quindi attorno ai *35 colpi* (ignorando l'informazione extra degli annunci di affondamento), oltre al vincolo banale di almeno 17 colpi. La cifra di 53 va letta come "quanto vale un tiro poco informato", non come soglia.

2.9.8 Che cosa portarsi dietro

- Le tre strategie sono tre livelli di uso dell'informazione: nessuno (casuale), locale (hunt and target), globale (bayesiana).
- La ricerca bayesiana non ha regole scritte a mano: massimizza l'utilità attesa rispetto a una distribuzione aggiornata. Il comportamento "intelligente" (cominciare dal centro, seguire l'orientamento della nave dopo due colpi) **emerge** dal calcolo.
- Ogni informazione disponibile va usata: tenere conto delle navi affondate vale da solo una dozzina di turni.
- È lo stesso schema delle applicazioni reali di *search theory*, come la ricerca del relitto del volo AF 447.

Riepilogo della lezione

- Le preferenze sono una relazione $\succsim$ su X; se è completa e transitiva è una *relazione di preferenza*, e da essa si derivano $\succ$ e $\sim$.
- **Teorema 1**: con X finito, $\succsim$ è rappresentabile da $u : X \to \mathbb{R}$ se e solo se è completa e transitiva. L'utilità è *ordinale*: invariante per trasformazioni strettamente crescenti, quindi i numeri non si interpretano.
- Con l'incertezza si passa alle *lotterie*. Una lotteria composta si riduce a una semplice moltiplicando le probabilità lungo i rami.
- **Teorema VNM**: $\succsim$ su $\mathcal{L}$ ammette utilità attesa $U(L) = \sum_i p_i u(x_i)$ se e solo se vale ordine di preferenza, continuità e indipendenza. L'utilità è *cardinale*: unica a meno di trasformazioni affini $aU + b$ con $a > 0$.
- Il *paradosso di Allais* mostra che le persone violano l'indipendenza: VNM è una teoria normativa, non descrittiva.
- Atteggiamento verso il rischio = curvatura di u: lineare $\to$ neutrale, concava $\to$ avverso, convessa $\to$ amante. Con $u(x) = \sqrt{x}$ la condivisione del rischio e l'assicurazione creano valore per entrambe le parti (premio $P \in [50, 75]$ nell'esempio).
- **Lab**: la ricerca bayesiana sceglie a ogni turno la cella di massima utilità attesa, aggiornando il posteriore con colpi a vuoto, a segno e affondamenti. Media misurata: 41–44 colpi, contro 61 di hunt and target e 95 del caso.

Prossima lezione: giochi non cooperativi in forma normale, *best response* ed equilibrio di Nash.